

BCS714A: DL MODULE 2

QUESTION 1:

Explain the need for regularization in deep learning with a suitable example and discuss overfitting.

ANS:

1. Introduction

Regularization is a method used to prevent **overfitting** in deep learning. Its main idea is to **limit the complexity of the model** so that it learns the important patterns in the training data instead of simply memorizing it.

A good model should not only perform well on training data but should also **generalize well to unseen data**.

2. What is Overfitting?

Overfitting occurs when a model learns the training data too well, including its **noise and unimportant details**, but performs poorly on new or unseen data.

Example

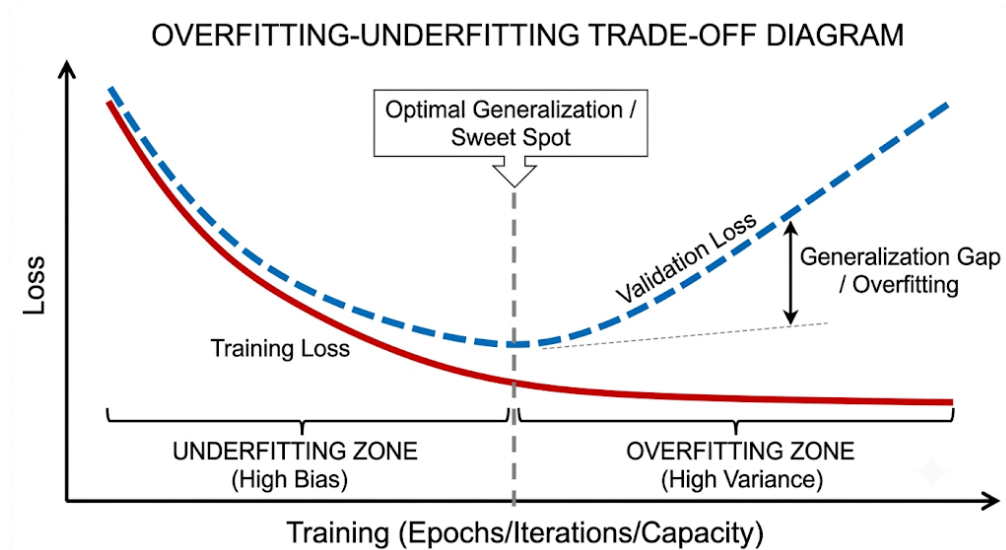
Suppose we train a neural network to classify images of **cats and dogs**.

```
Training Images
  ↓
Neural Network
  ↓
Learns useful features + noise
  ↓
Very high training accuracy
  ↓
Poor performance on new images
```

The model may **memorize specific training images** instead of learning general characteristics of cats and dogs.

Training vs Validation Behaviour

A typical overfitting situation is:



- **Training loss** continues to decrease.
- **Validation loss** initially decreases but later starts increasing.
- The point where validation performance starts getting worse indicates the beginning of **overfitting**.

3. Why is Regularization Needed?

Deep neural networks can have a **large number of parameters**, making them highly flexible.

A highly complex model can fit even the **noise in training data**.

Therefore:

High Model Complexity → Overfitting

Regularization attempts to control this complexity:

Regularization → Simpler Model → Better Generalization

The objective is not simply to obtain low training error, but to obtain good performance on **unseen data**.

4. Parameter Norm Penalties

One important regularization method is **parameter norm penalty**.

The parameters of a neural network are its **weights and biases**. Large weights can make the model highly sensitive to small changes in the input and increase the risk of overfitting.

Therefore, a penalty is added to the loss function to discourage excessively large parameters.

Regularized Objective

$$J_{\text{regularized}} = J_{\text{data}} + \lambda \Omega(\theta)$$

where:

- J_{data} = original loss for fitting the training data
- θ = model parameters
- $\Omega(\theta)$ = penalty based on parameter size
- λ = regularization strength

Thus, the model is encouraged to:

fit the data well + keep parameters small.

 BCS714A-module-2-pdf(1)

5. Suitable Example

Consider a simple model:

$$y = w_1 x$$

Without regularization

Suppose:

$$w_1 = 100$$

Even a tiny change in x can cause a large change in the output. Therefore, the model can become highly sensitive to noise.

With regularization

Regularization penalizes such large weights and encourages smaller values of w_1 .

Without Regularization

↓

Large Weights

↓

High Model Complexity

↓

Sensitive to Noise

↓

Overfitting

With Regularization

↓

Smaller Weights

↓

Simpler Model

↓

Less Sensitive to Noise

↓

Better Generalization

This illustrates why controlling parameter size helps reduce overfitting.

6. Other Regularization Methods

Deep learning uses several approaches to reduce overfitting:

1. Parameter Norm Penalties

Penalize large model parameters and encourage a **simpler model**.

2. Dataset Augmentation

Increase the effective training data by creating modified versions of existing examples.

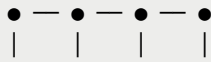
3. Noise Injection

Adding small random noise can prevent the model from becoming excessively sensitive to individual training examples.

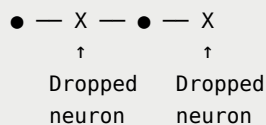
4. Dropout

Dropout is a regularization technique in which random hidden neurons are temporarily **dropped (set to zero)** during training.

Before Dropout:



After Dropout:



This prevents the network from relying too heavily on particular neurons and improves generalization.

5. Early Stopping

Early stopping monitors validation performance and stops training when the validation error stops improving.

The parameters corresponding to the **lowest validation error** are retained as the final model.

QUESTION 2:

Write the mathematical formulation for the following parameter norm penalties:

- i) Limiting model capacity
- ii) L^2 parameter regularization
- iii) L^1 regularization

For each of the above, give the expression for the norm penalty term $\Omega(\theta)$ and explain in each case, the effect of adding these penalties to the learning models. Also write the equations for the following giving explanation of the terms there-in.

- I. Closed form of Ridge regression
- II. Soft Thresholding rule for lasso slope solution.

ANS:

1. Introduction

Parameter norm penalties are used to control the complexity of a deep learning model and reduce **overfitting**. The basic idea is to add a penalty to the loss function so that the model is encouraged to use **smaller parameter values**. Large weights can make a model highly sensitive to small changes in the input and can cause overfitting.

The general regularized objective is:

$$J_{\text{reg}}(\theta) = J(\theta) + \lambda\Omega(\theta)$$

where:

- $J(\theta)$ = original loss/data-fitting term
- θ = model parameters
- $\Omega(\theta)$ = norm penalty
- λ = regularization coefficient

2. Limiting Model Capacity

Instead of directly adding a penalty, model capacity can be limited by imposing a **constraint on the parameter norm**.

Mathematical formulation

$$\min_{\theta} J(\theta)$$

subject to

$$\|\theta\|_p \leq r$$

where:

- θ = model parameters
- $J(\theta)$ = loss function
- $\|\theta\|_p = L^p$ -norm of parameters
- r = maximum allowed parameter magnitude.

Thus, the corresponding constraint is:

$$\Omega(\theta) = \|\theta\|_p$$

Effect

- Prevents parameters from becoming excessively large.
- **Reduces model complexity.**
- Prevents the model from fitting noise in the training data.
- Improves **generalization** to unseen data.

The notes emphasize that **large weights correspond to a more complex model**, whereas smaller weights give smoother predictions and reduce sensitivity to noise.

3. L² Parameter Regularization

In L² regularization, the squared magnitude of the parameters is penalized.

Norm penalty

$$\Omega(\theta) = \frac{1}{2} \|\theta\|_2^2$$

or equivalently,

$$\Omega(\theta) = \frac{1}{2} \sum_i \theta_i^2$$

Therefore, the regularized objective becomes:

$$J_{\text{reg}}(\theta) = J(\theta) + \frac{\lambda}{2} \sum_i \theta_i^2$$

Effect of L² penalty

The penalty discourages **large weights** and pulls parameters toward zero.

Without L²

Large weights

↓

High complexity

↓

Overfitting

With L²

Small weights

↓

Lower complexity

↓

Better generalization

L^2 is also called **weight decay**. It performs **soft shrinkage**: most parameters remain non-zero but their magnitudes are reduced.

4. L^1 Regularization

In L^1 regularization, the absolute values of parameters are penalized.

Norm penalty

$$\Omega(\theta) = \|\theta\|_1$$

$$\Omega(\theta) = \sum_i |\theta_i|$$

Therefore:

$$J_{\text{reg}}(\theta) = J(\theta) + \lambda \sum_i |\theta_i|$$

Effect of L^1 penalty

Unlike L^2 , L^1 regularization can force some parameters **exactly to zero**.

Therefore, it produces a **sparse model**:

Before L^1 :

[2.4, -1.2, 0.8, 0.3, -0.6]

After L^1 :

[2.1, 0, 0, 0, -0.4]

5. L² vs L¹ Regularization

Feature	L ² Regularization	L ¹ Regularization
Penalty	$\frac{1}{2} \sum \theta_i^2$	$\lambda \sum$
Effect	Shrinks weights	Drives some weights to zero
Sparsity	Generally no exact zeros	Produces sparse solutions
Nature	Soft shrinkage	Hard selection
Also called	Weight decay	Lasso regularization

6. Closed Form of Ridge Regression

Ridge regression is linear regression with an L^2 penalty.

Its objective is:

$$J(w) = \frac{1}{2} \|Xw - y\|_2^2 + \frac{\lambda}{2} \|w\|_2^2$$

To minimize it, differentiate with respect to w :

$$X^T(Xw - y) + \lambda w = 0$$

$$X^T X w - X^T y + \lambda w = 0$$

$$(X^T X + \lambda I)w = X^T y$$

Therefore, the **closed-form Ridge regression solution** is:

$$w^* = (X^T X + \lambda I)^{-1} X^T y$$

where:

- X = input/design matrix
- y = target vector
- w = model parameters
- $\lambda = L^2$ regularization strength
- I = identity matrix.

The term λI shrinks the parameters and makes the solution more stable.

7. Soft Thresholding Rule for Lasso

Lasso regression uses an L^1 penalty:

$$J(w) = \frac{1}{2} \|Xw - y\|_2^2 + \lambda \|w\|_1$$

The key operation in solving the Lasso problem is the **soft-thresholding operator**.

Soft-thresholding rule

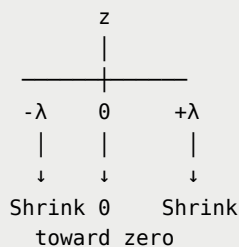
$$S(z, \lambda) = \begin{cases} z - \lambda, & z > \lambda \\ 0, & |z| \leq \lambda \\ z + \lambda, & z < -\lambda \end{cases}$$

Equivalently,

$$S(z, \lambda) = \text{sign}(z) \max(|z| - \lambda, 0)$$

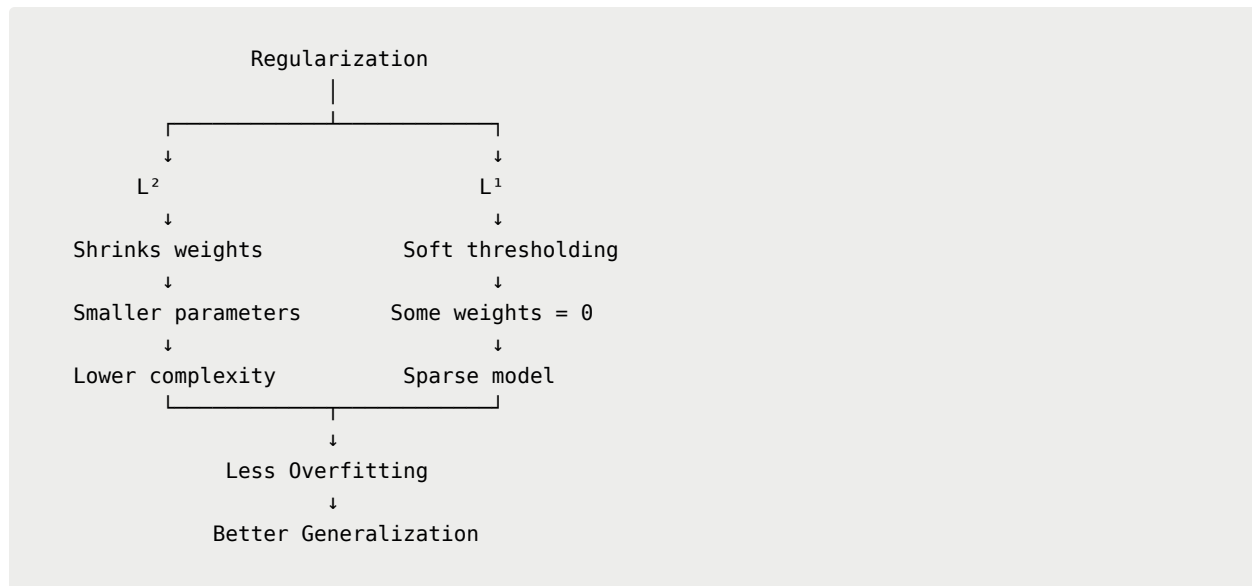
Interpretation

- If $z > \lambda$, reduce it by λ .
- If $z < -\lambda$, increase it toward zero by λ .
- If $|z| \leq \lambda$, set it **exactly to zero**.



Thus, the Lasso solution automatically removes sufficiently small coefficients, producing **sparsity**.

8. Overall Effect



QUESTION 3:

Explain data augmentation and discuss why it is effective in object recognition/generalization.

ANS:

1. Definition

Data augmentation is a technique in which new training examples are created by applying suitable **transformations to existing training data** and adding the transformed examples to the training set.

It is especially useful when the available amount of real training data is limited.

2. Basic Idea

The main objective is to effectively increase the size and variety of the training dataset.

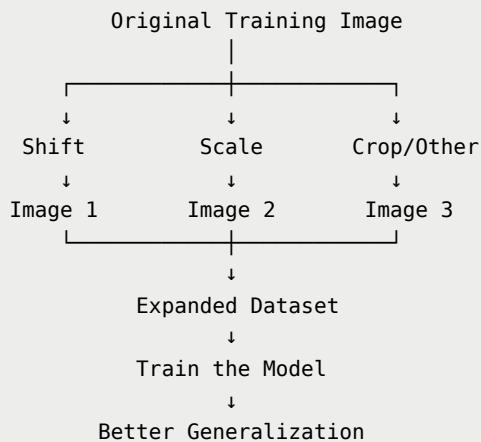
For a training pair (x, y) , where x is the input and y is its label, we apply a transformation T :

$$x' = T(x)$$

If the transformation does not change the class:

$$(x, y) \rightarrow (T(x), y)$$

Thus, several new training examples can be generated from one original example.



The transformations must preserve the correct class label.

3. Why Data Augmentation is Effective for Object Recognition

Data augmentation is particularly effective for **classification/object recognition** because a classifier should recognize an object despite many changes in its appearance.

For example, an object may appear:

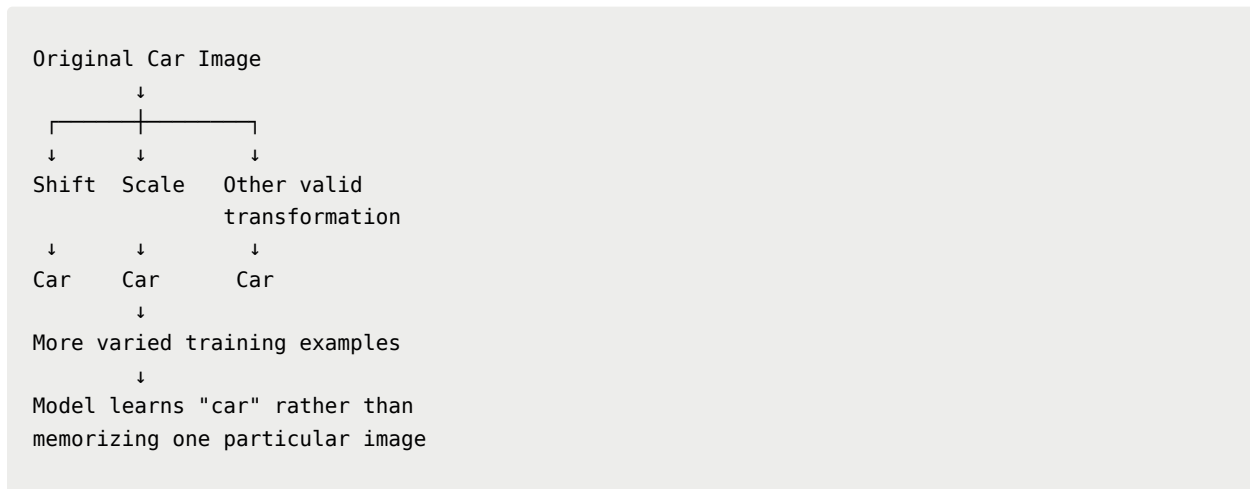
- At a different **position**
- At a different **scale**
- With slightly different orientation
- With other variations that do not change its identity

The classifier therefore needs to learn **invariance to a wide variety of transformations**. Data augmentation exposes the model to these variations during

training.

Example

Suppose the training set contains only one image of a **car**.



Therefore, when a new car image appears in a different position or appearance, the model is more likely to recognize it correctly.

4. Effect on Generalization

The main purpose of data augmentation is to improve **generalization**.

Without augmentation:

Limited Training Data → Memorization → Overfitting

With augmentation:

More Varied Training Examples → Learn General Patterns → Better Generalization

The model is exposed to different valid forms of the same object instead of repeatedly seeing exactly the same training examples.

Thus, augmentation helps the model learn the **underlying characteristics of an object** rather than depending on the exact appearance of a particular training image.

5. Important Condition: Transformations Must Be Valid

Not every transformation is safe. The transformation must preserve the meaning or class of the original input.

For example, in **text/character recognition**:

```
'b'  —flip—>  'd'  
'6'  —flip—>  '9'
```

Here, flipping changes the identity of the character. If such transformed images are given the original labels, they can **confuse the model instead of improving it**.

Therefore:

Valid transformation $\Rightarrow$ same correct label

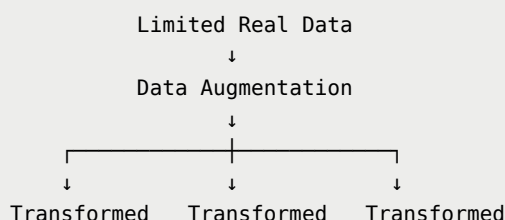
6. Why It Improves Generalization

Data augmentation provides the following benefits:

1. **Increases effective training data** without collecting the same amount of new real data.
2. Exposes the model to **different variations** of objects.
3. Encourages the classifier to become **invariant to valid transformations**.
4. Reduces dependence on individual training examples.
5. Helps prevent **overfitting**.
6. Improves performance on **unseen examples**.

7. Significance in Deep Learning

Deep learning models often have a large number of parameters and can therefore easily memorize limited training data. Data augmentation provides additional variation during training and acts as a useful form of **regularization**.





QUESTION 4:

Differentiate between Batch Gradient Descent and Mini-batch Gradient Descent.

ANS:

1. Introduction

Gradient Descent is an optimization method used to train neural networks by updating the **weights and biases** in the direction that reduces the loss.

The main difference between **Batch Gradient Descent** and **Mini-batch Gradient Descent** is the **amount of training data used to calculate the gradient before updating the weights**.

2. Batch Gradient Descent

In **Batch Gradient Descent**, the **entire training dataset** is used to calculate the loss and gradient before performing one parameter update.

Procedure

```
graph TD; A[Entire Training Dataset] --> B[Compute Gradient]; B --> C[Update Weights]; C --> D[Next Batch]; D --> E[Repeat]
```

The procedure for Batch Gradient Descent involves using the entire training dataset to compute the gradient, update the weights, and then repeating the process for the next batch.

For N training examples:

$$\theta \leftarrow \theta - \eta \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} L_i$$

where:

- θ = model parameters
- η = learning rate
- L_i = loss for the i^{th} training example
- N = total number of training examples.

Example

If the dataset contains **1 million images**, Batch Gradient Descent calculates the loss and gradient using **all 1 million images** before updating the weights. [BCS714A-module-2-pdf\(1\)](#)

3. Mini-batch Gradient Descent

In **Mini-batch Gradient Descent**, the training dataset is divided into **small batches**, and the parameters are updated after processing each mini-batch.

Typical batch sizes include:

32, 64, 128

Procedure

Training Dataset

↓

B1	B2	B3	B4	...
----	----	----	----	-----

↓

Compute Gradient

↓

Update Weights

↓

Next Mini-batch

↓

Repeat

For a mini-batch of size m :

$$\theta \leftarrow \theta - \eta \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} L_i$$

Example

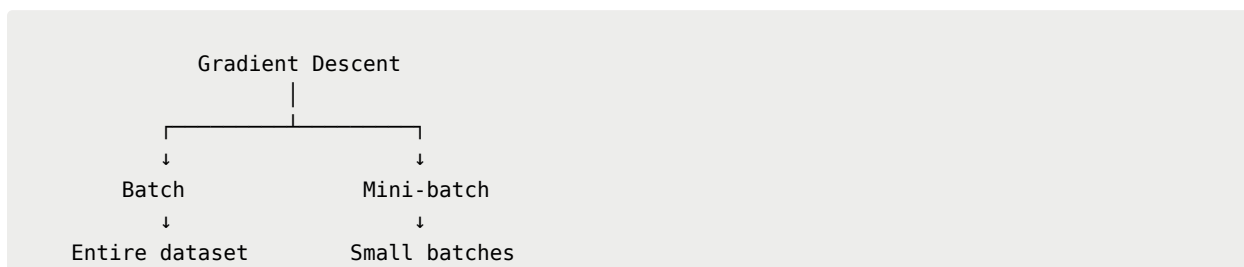
For 1 million images with a batch size of 64, the weights are updated after every 64 images.

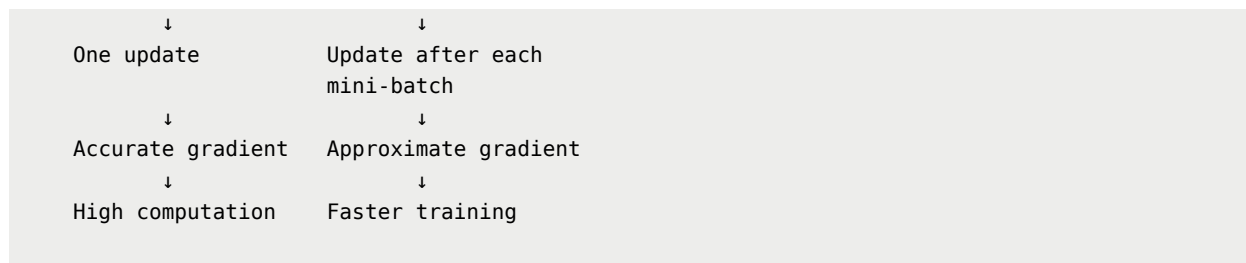
4. Differences

Feature	Batch Gradient Descent	Mini-batch Gradient Descent
Data used per update	Entire dataset	Small subset of dataset
Batch size	N	Usually 32, 64, 128, etc.
Number of updates	One update after processing the complete dataset	Multiple updates within one pass through the dataset
Gradient	More accurate/deterministic	Approximate and slightly noisy
Training speed	Slower for very large datasets	Faster
Memory requirement	High	Lower
GPU suitability	Less practical for very large datasets	Works well with GPUs
Stability	More stable updates	Somewhat noisy updates
Usage in deep learning	Less common for large datasets	Most commonly used

The notes specifically identify mini-batch training as faster than full-batch training, more stable than using individual samples, and well suited to **GPU parallel processing**.

5. Overall Comparison





Important point

Stochastic Gradient Descent (SGD) can be considered a special case of mini-batch gradient descent where:

$$\boxed{\text{Batch size} = 1}$$

Thus:

$$\boxed{\text{Batch GD} \rightarrow \text{Mini-batch GD} \rightarrow \text{SGD}}$$

as the amount of data used per update decreases. [BCS714A-module-2-pdf\(1\)](#)

QUESTION 5:

Write the algorithm for **Stochastic Gradient Descent with Momentum** and explain the core idea of momentum and how it helps optimization.

ANS:

1. Definition

Stochastic Gradient Descent (SGD) updates the model parameters using a small random subset of training data. **Momentum** improves SGD by remembering the direction of previous updates and using it along with the current gradient.

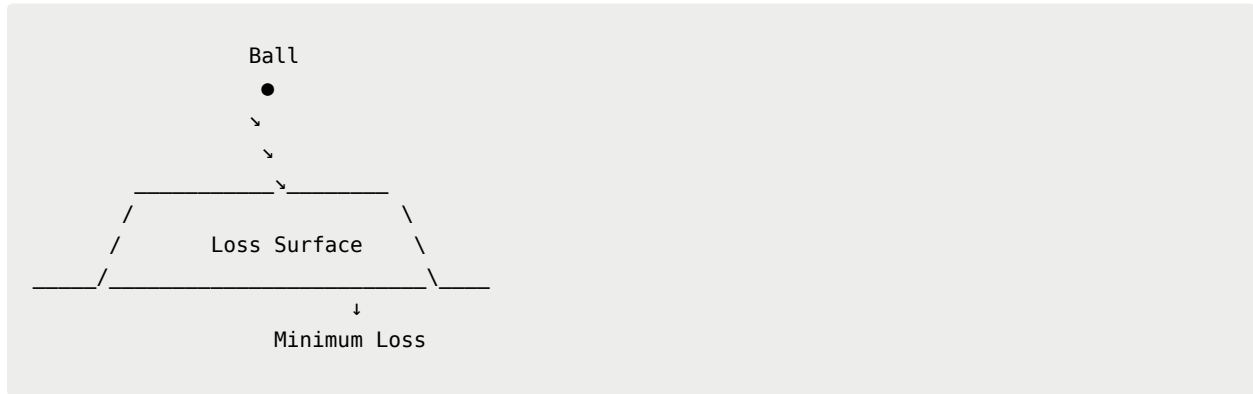
Thus, momentum helps the optimizer **converge faster and reduce oscillations**.

2. Core Idea of Momentum

In ordinary SGD, the parameter update depends only on the **current gradient**. If the gradient changes direction frequently, the optimization process may oscillate.

Momentum maintains a velocity based on past gradients. It gives the optimizer a continued push in the direction in which it has been moving.

A useful analogy is a **ball rolling down a hill**:



The previous movement gives the ball momentum, allowing it to move through **small dips, bumps and flat regions** more smoothly.

3. Mathematical Formulation

Let:

- θ_t = parameters at iteration t
- g_t = gradient estimated using the current training sample/mini-batch
- η = learning rate
- v_t = velocity/momentum term
- β = momentum coefficient

The momentum update is:

$$v_t = \beta v_{t-1} - \eta g_t$$

Then the parameters are updated as:

$$\theta_t = \theta_{t-1} + v_t$$

The term βv_{t-1} carries information from the **previous update**, while $-\eta g_t$ represents the effect of the **current gradient**.

4. Algorithm: SGD with Momentum

Algorithm

1. **Initialize** model parameters θ and velocity $v = 0$.
2. Choose the **learning rate** η and momentum coefficient β .
3. Select a **random training example or mini-batch**.
4. Perform forward propagation and calculate the **loss**.
5. Compute the gradient:

$$g_t = \nabla_{\theta} J(\theta_t)$$

6. Update the velocity:

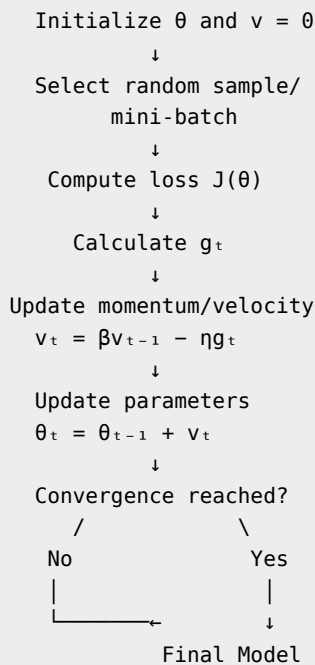
$$v_t = \beta v_{t-1} - \eta g_t$$

7. Update the parameters:

$$\theta_t = \theta_{t-1} + v_t$$

8. Repeat Steps 3–7 until the model has **converged** or trained sufficiently.

Flow diagram



5. How Momentum Helps Optimization

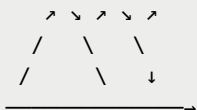
1. Faster Convergence

Momentum accumulates useful directions from previous gradients, allowing the optimizer to move **faster toward the minimum** instead of relying entirely on the current gradient.

2. Reduces Oscillations

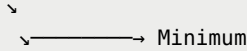
When gradients repeatedly change direction, momentum smooths the updates and reduces **zig-zag movement**.

Without Momentum:



With Momentum:





3. Moves Through Flat Regions

In regions where the current gradient is small, accumulated momentum can continue moving the parameters in a useful direction.

4. Helps Overcome Small Obstacles

Momentum can help the optimizer move past **small dips and irregularities** in the loss surface rather than becoming slowed down by them.

5. Smoother Training

Because past gradients are incorporated, parameter updates become **less sensitive to individual noisy gradient estimates**.

6. SGD vs SGD with Momentum

Feature	SGD	SGD with Momentum
Uses current gradient	Yes	Yes
Uses previous updates	No	Yes
Oscillations	Higher	Reduced
Convergence	Slower	Faster
Flat regions	May progress slowly	Momentum helps continue movement
Extra parameter	Learning rate	Learning rate + momentum coefficient

QUESTION 6:

Describe the key mechanics of the Adam optimization algorithm.

ANS:

Adam Optimization Algorithm

1. Introduction

Adam (Adaptive Moment Estimation) is an **adaptive learning-rate optimization algorithm** widely used in deep learning. It combines the advantages of **Momentum** and **RMSProp**.

- **Momentum** uses the first-order moment of gradients to smooth updates.
 - **RMSProp** uses the second-order moment, i.e., squared gradients, to adapt the learning rate for each parameter.
 - Adam additionally uses **bias correction** to improve the accuracy of these estimates during early training.
-

2. Key Mechanics of Adam

A. First Moment — Momentum

Adam maintains an **exponentially weighted moving average of past gradients**:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

where:

- g_t = current gradient
- m_t = first moment estimate
- β_1 = decay rate for the first moment.

This provides the **momentum effect**, smoothing gradient updates and reducing oscillations.

 BCS714A-module-2-pdf(1)

B. Second Moment — RMSProp

Adam also maintains an exponentially weighted moving average of the **squared gradients**:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

where:

- v_t = second moment estimate
- β_2 = decay rate for the second moment.

This allows Adam to **adapt the learning rate individually for each parameter**, depending on the magnitude of its gradients.

 BCS714A-module-2-pdf(1)

3. Bias Correction

Initially,

$$m_0 = 0, \quad v_0 = 0$$

Therefore, the moment estimates can be biased toward zero during the first few iterations.

Adam corrects this bias using:

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

The corrected values provide **more accurate moment estimates during early training** and improve training stability.  BCS714A-module-2-pdf(1)

4. Parameter Update

After calculating the bias-corrected moments, the parameters are updated as:

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

where:

- θ_t = updated parameters
- η = learning rate
- $\hat{m}_t$ = bias-corrected first moment
- $\hat{v}_t$ = bias-corrected second moment
- ϵ = small constant added for numerical stability.

5. Adam Algorithm

```
Initialize  $\theta, m = 0, v = 0$ 
  ↓
Select training batch
  ↓
Compute gradient  $g_t$ 
  ↓
First moment (Momentum)
 $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ 
  ↓
Second moment (RMSProp)
```

```


$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

    ↓
    Bias correction

$$\hat{m}_t = m_t / (1-\beta_1^t)$$


$$\hat{v}_t = v_t / (1-\beta_2^t)$$

    ↓
    Update parameters

$$\theta_t = \theta_{t-1} - \eta \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$$

    ↓
    Repeat until
    convergence

```

6. Why Adam is Effective

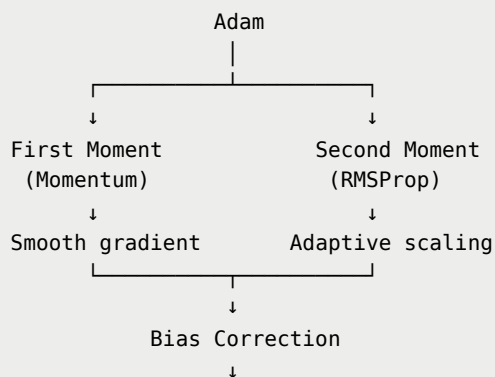
Adam combines two important ideas:

Adam = Momentum + RMSProp + Bias Correction

Advantages

1. **Faster convergence** — momentum accelerates movement in useful directions.
2. **Reduced oscillations** — past gradients smooth the updates.
3. **Adaptive learning rates** — each parameter gets an update scaled according to its gradient magnitude.
4. **Better early training stability** — bias correction provides more accurate initial moment estimates.
5. It is particularly effective for **complex, high-dimensional neural networks**.

7. Overall Working



Parameter Update
↓
Faster, stable training

QUESTION 7:

Explain parameter initialization strategies and discuss Xavier and He initialization methods.

ANS:

1. Introduction

Parameter initialization is the process of choosing the initial values of the **weights and biases** of a neural network before training begins.

Deep learning models are trained using **iterative optimization algorithms** such as gradient descent, so the initial parameter values strongly affect **convergence, numerical stability and the quality of the final solution**.

2. Why Parameter Initialization is Important

A good initialization should:

1. Help the network **converge quickly**.
2. Prevent **vanishing gradients**.
3. Prevent **exploding gradients**.
4. Maintain a reasonable **activation variance** across layers.
5. Maintain a reasonable **gradient variance** during backpropagation.
6. **Break symmetry**, so different neurons learn different features.

Effect of poor initialization

Poor Initialization
↓
Vanishing / Exploding Gradients
↓
Slow or No Convergence
↓

Poor Training
↓
Poor Generalization

If all neurons start with identical parameters, they receive the same inputs and learn the same features. Hence, **symmetry breaking** is essential.

3. Common Parameter Initialization Strategies

1. Random Initialization

Weights are initialized randomly using a **Gaussian or uniform distribution**.

- Biases are commonly initialized to zero or a small constant.
- The scale of the random distribution is important.
- **Too large** → exploding activations/gradients.
- **Too small** → vanishing activations/gradients.

2. Xavier (Glorot) Initialization

Designed to maintain activation and gradient variance across layers and is suitable for **sigmoid/tanh activations**.

3. He Initialization

Designed specifically for **ReLU activations**.

4. Orthogonal Initialization

Uses **random orthogonal matrices** so that different units initially have independent directions.

5. Sparse Initialization

Only a few weights connected to each neuron are initially non-zero.

6. Data-Based Initialization

Uses statistics of the training data to initialize parameters such as biases.

7. Pretraining

Weights can be initialized using parameters learned from another model or a previous training procedure.

4. Xavier (Glorot) Initialization

Xavier initialization, also called Glorot initialization, scales the weights according to the number of **input** and **output connections**.

Let:

- n_{in} = number of input connections (**fan-in**)
- n_{out} = number of output connections (**fan-out**)

A commonly used Xavier normal formulation is:

$$W \sim N\left(0, \frac{2}{n_{in} + n_{out}}\right)$$

For a uniform distribution:

$$W \sim U\left(-\sqrt{\frac{6}{n_{in} + n_{out}}}, +\sqrt{\frac{6}{n_{in} + n_{out}}}\right)$$

Purpose

Xavier initialization aims to maintain a **similar variance of activations and gradients across layers**, preventing them from becoming extremely small or large.

Suitable activation

Xavier → Sigmoid / Tanh

The notes specifically identify Xavier initialization as suitable for **sigmoid/tanh activations**.

 BCS714A-module-2-pdf(1)

5. He Initialization

He initialization was designed specifically for networks using the **ReLU** activation function.

The weights are initialized with variance:

$$\text{Var}(W) = \frac{2}{n_{in}}$$

Therefore, the commonly used normal form is:

$$W \sim N\left(0, \frac{2}{n_{in}}\right)$$

where n_{in} is the number of input connections.

Purpose

ReLU sets negative activations to zero. He initialization compensates for this behaviour by using the factor 2 so that the activation variance remains appropriately scaled through the network.

Suitable activation

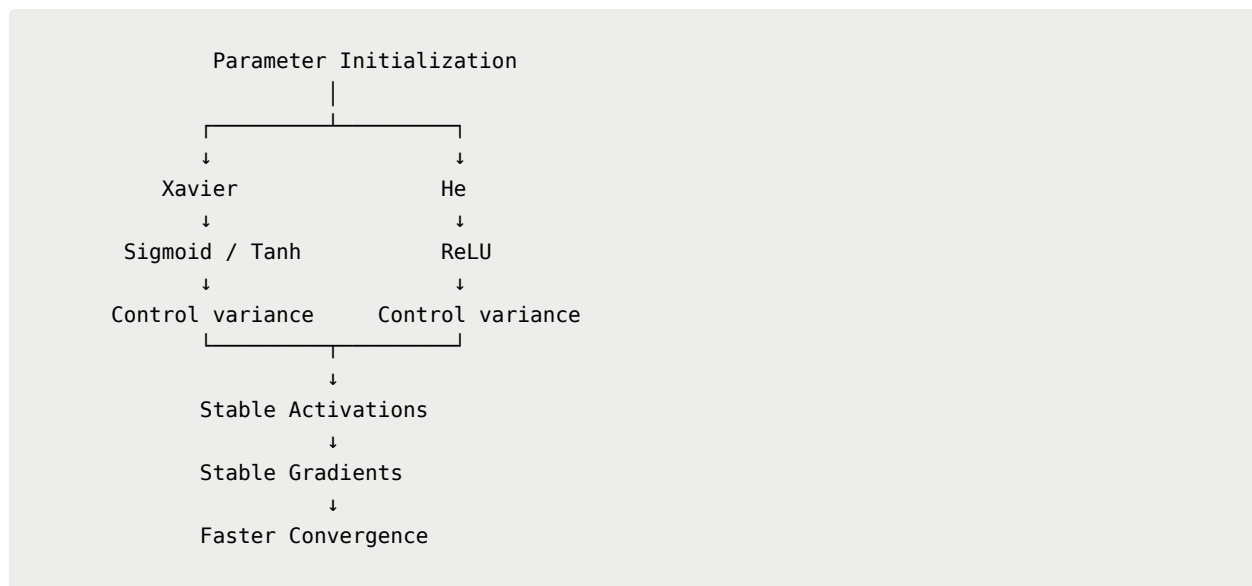
$$\text{He} \rightarrow \text{ReLU}$$

The notes explicitly state that He initialization is **specifically designed for ReLU activations** and uses variance scaled by $2/n_{in}$.  BCS714A-module-2-pdf(1)

6. Xavier vs He Initialization

Feature	Xavier (Glorot)	He Initialization	
Main purpose	Maintain activation/gradient variance	Maintain activation/gradient variance with ReLU	
Variance	$\frac{2}{n_{in} + n_{out}}$	$\frac{2}{n_{in}}$	
Depends on	Fan-in and fan-out	Fan-in	
Best suited for	Sigmoid / Tanh	ReLU	
Main benefit	Stable signal propagation	Effective signal propagation in ReLU networks	

7. Overall Concept



QUESTION 8:

ANS:

Explain noise robustness as a regularization technique in deep learning. Discuss Dropout in detail and explain Early Stopping as a regularization technique with suitable algorithms/examples.

ANS:

1. Introduction

Regularization is used in deep learning to prevent **overfitting** and improve the model's ability to **generalize to unseen data**.

Three important regularization techniques are:

1. **Noise Robustness / Noise Injection**
2. **Dropout**
3. **Early Stopping**

2. Noise Robustness as Regularization

Noise robustness refers to making a neural network less sensitive to small variations or noise in its inputs or internal representations.

Noise Injection

A small amount of **random noise** can be added to the input during training.

```
Original Input
↓
Add Small Random Noise
↓
Noisy Input
↓
Neural Network
↓
Prediction
```

Adding very small random noise to the inputs is mathematically similar to adding a **penalty on the model parameters**. This penalty discourages excessively large weights and therefore acts as a form of regularization.

Noise in Hidden Layers

Noise need not be restricted to the input. Adding noise to **hidden layers** can be even more effective because it forces the network to learn representations that are **robust to variations** rather than depending heavily on particular activations. This improves generalization.

Example

If a network is trained to recognize a handwritten digit, slightly changing the pixel values should not cause the predicted digit to change.

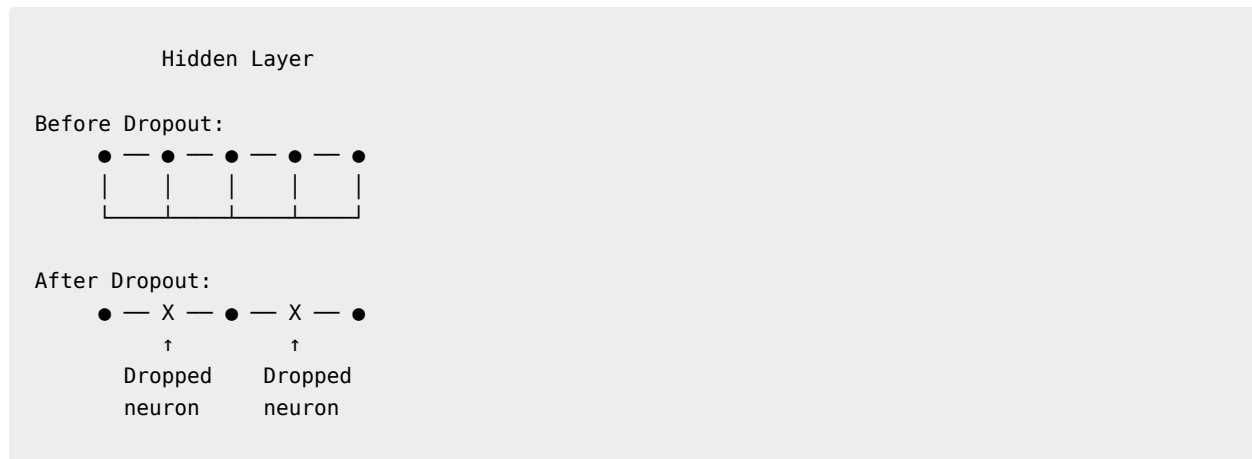
Noise Injection → Robust Features → Better Generalization

3. Dropout

Dropout is a regularization technique in which random hidden neurons are **dropped out (set to zero)** during training.

The main purpose is to prevent the network from becoming overly dependent on particular neurons.

Diagram



Here, **X** represents a neuron that has been temporarily set to zero.

Working of Dropout

1. Select a **dropout probability**.
2. During each training iteration, randomly select hidden units.
3. Set the selected units to **zero**.
4. Perform forward and backward propagation using the remaining units.
5. In the next iteration, a different set of neurons may be dropped.
6. Continue training until convergence.

For example, with a dropout probability of **0.5**, approximately half of the selected neurons are dropped during a particular training pass.

Why Dropout Works

Without dropout:

Network → Dependence on Specific Neurons → Overfitting

With dropout:

Random Neurons Removed → Less Co-adaptation → More Robust Network → Better Generalization

Dropout can therefore be viewed as a special case of **noise injection into hidden units**.

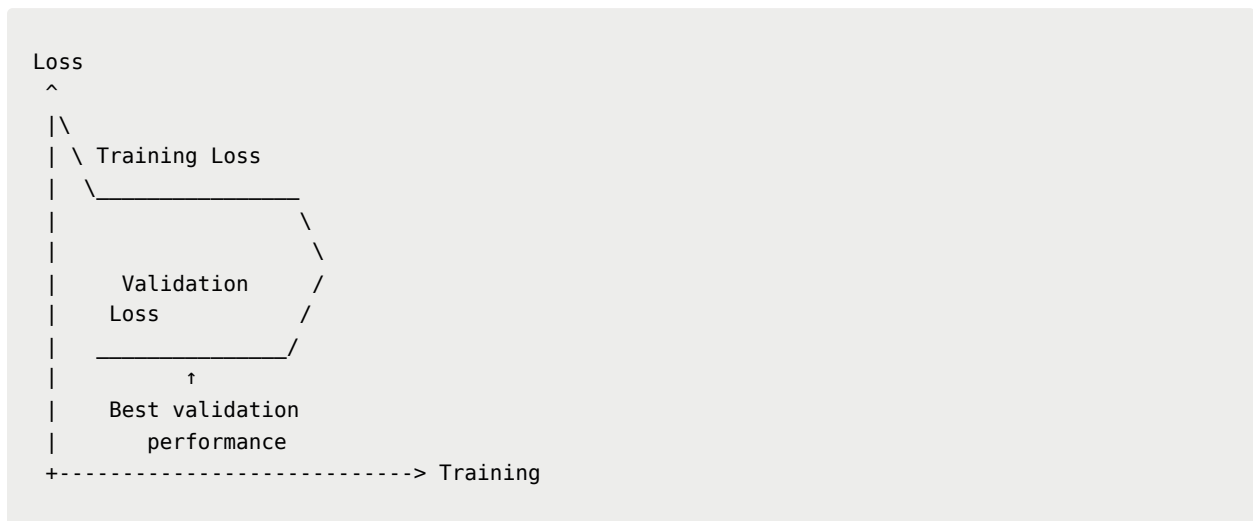
4. Early Stopping

Early stopping is a regularization technique that stops training when performance on a **validation set** stops improving.

During training:

- **Training loss** generally continues to decrease.
- **Validation loss** initially decreases.
- After some point, validation loss starts increasing because the model begins to **overfit**.

Diagram



The model should be selected at the point where the **validation error is minimum**, rather than at the final training iteration.

5. Early Stopping Algorithm

Let:

- v = best validation error so far
- θ^* = best parameters
- i^* = iteration at which the best parameters were obtained
- p = **patience**
- j = number of consecutive checks without improvement
- n = number of training steps between validation checks.

Algorithm

1. Initialize:

$$v \leftarrow \infty$$

$$\theta^* \leftarrow \theta, \quad i^* \leftarrow 0$$

$$j \leftarrow 0$$
2. While $j < p$:
 - a. Train the model for n steps/epochs.
 - b. Update the training-step counter:

$$i \leftarrow i + n$$
 - c. Calculate the **validation error**.
 - d. If validation error improves:

$$\text{if } E_{\text{val}} < v$$

then:

$$v \leftarrow E_{\text{val}}$$

$$\theta^* \leftarrow \theta$$

$$i^* \leftarrow i$$

and reset:

$$j \leftarrow 0$$

e. Otherwise:

$$j \leftarrow j + 1$$

3. When:

$$j = p$$

stop training.

4. Return:

$$\boxed{\theta^*}$$

the parameters that produced the **lowest validation error**, rather than the parameters from the final iteration.

6. Suitable Example of Early Stopping

Suppose the validation errors during training are:

Epoch	Validation Error
10	0.40

Epoch	Validation Error
20	0.30
30	0.20
40	0.23
50	0.25
60	0.27

The best validation error is **0.20 at epoch 30**.

Therefore, even though training continues to improve after epoch 30, the model should retain the parameters from **epoch 30**.

If the specified **patience** is 3 checks and no improvement occurs for three consecutive checks, training terminates and the stored best parameters are returned.

7. Comparison

Technique	Basic Idea	Main Effect
Noise Robustness	Add small random noise to inputs/hidden units	Makes model robust and reduces overfitting
Dropout	Randomly set hidden neurons to zero during training	Prevents dependence on particular neurons
Early Stopping	Stop when validation performance stops improving	Prevents further training from causing overfitting

QUESTION 9:

Explain adaptive gradient-based optimization methods in deep learning. Discuss Nesterov Accelerated Gradient, AdaGrad and RMSProp, explaining their working, mathematical formulations and advantages.

ANS:

1. Introduction

Adaptive gradient-based optimization methods improve gradient descent by adjusting the **step size or learning rate for individual parameters** according to the behavior of their gradients.

Different directions in parameter space can have different sensitivities. Therefore, using the same learning rate for every parameter may be inefficient. Adaptive methods attempt to take **smaller steps in steep directions and larger steps in flatter directions**.

The important methods are:

1. **Nesterov Accelerated Gradient (NAG)**
 2. **AdaGrad**
 3. **RMSProp**
-

2. Nesterov Accelerated Gradient (NAG)

Nesterov Accelerated Gradient is an improved version of momentum-based gradient descent. Its key idea is to perform a **look-ahead step** using the current momentum and then calculate the gradient at that future position.

Basic idea

```
Current position
  ↓
Move temporarily in direction of momentum
  ↓
Look-ahead position
  ↓
Calculate gradient there
  ↓
Correct the update
  ↓
New parameter position
```

Unlike classical momentum, which calculates the gradient at the **current position**, NAG first predicts where the parameters are going and evaluates the gradient there.

Mathematical formulation

Let:

- θ_t = parameters at iteration t
- v_t = velocity
- η = learning rate
- μ = momentum coefficient
- $g(\theta)$ = gradient.

First calculate the look-ahead position:

$$\theta_{\text{look}} = \theta_{t-1} + \mu v_{t-1}$$

Calculate the gradient at this position:

$$g_t = \nabla J(\theta_{\text{look}})$$

Update velocity:

$$v_t = \mu v_{t-1} - \eta g_t$$

Update parameters:

$$\theta_t = \theta_{t-1} + v_t$$

Advantages

- **Faster convergence**, especially on flat or curved loss surfaces.
- Reduces **oscillations**.
- The look-ahead gradient provides better anticipation of future movement.
- More responsive than classical momentum when approaching minima.

3. AdaGrad

AdaGrad (Adaptive Gradient) adapts the learning rate **individually for each parameter** using the accumulated history of squared gradients. BCS714A-module-2-pdf(1)

Working

For every parameter, AdaGrad maintains a running sum of its past squared gradients.

Let:

$$g_t = \nabla_{\theta} J(\theta_t)$$

The accumulated squared gradient is:

$$G_t = G_{t-1} + g_t^2$$

The parameter update is:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t} + \epsilon} g_t$$

where:

- G_t = accumulated squared gradients
- η = initial learning rate
- ϵ = small constant for numerical stability.

Interpretation

If a parameter frequently receives **large gradients**, its accumulated G_t becomes large, so its effective learning rate decreases.

If a parameter has **small or infrequent gradients**, its learning rate decreases more slowly.

```
Large gradients
  ↓
Large accumulated G
  ↓
Smaller learning rate

Small/infrequent gradients
  ↓
Smaller accumulated G
  ↓
Relatively larger learning rate
```

This makes AdaGrad particularly effective for **sparse-data problems**, such as some NLP and recommendation-system applications.

Advantages

- Automatically provides a **different learning rate for each parameter**.
- Reduces updates in frequently changing/steep directions.
- Allows relatively larger updates in flatter directions.
- Particularly useful for **sparse features**.

Limitation

The major problem is that GtG_t **continuously accumulates** squared gradients. Consequently, the effective learning rate may become extremely small, causing training to **slow down or stop prematurely**.

4. RMSProp

RMSProp (Root Mean Square Propagation) was developed to overcome the excessive learning-rate decay problem of AdaGrad. Instead of accumulating **all historical squared gradients**, RMSProp uses an **exponentially weighted moving average (EWMA)**, so recent gradients have greater influence.

BCS714A-module-2-pdf(1)

Working

The moving average of squared gradients is:

$$S_t = \beta S_{t-1} + (1 - \beta) g_t^2$$

The parameter update is:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{S_t} + \epsilon} g_t$$

where:

- S_t = exponentially weighted average of squared gradients
- β = decay rate
- η = learning rate
- ϵ = small constant.

Key Difference from AdaGrad

```
AdaGrad
Past squared gradients
  ↓
Continuous accumulation
  ↓
Learning rate keeps shrinking
  ↓
May eventually stop learning

RMSProp
Recent squared gradients
  ↓
Exponential moving average
  ↓
Old information fades
  ↓
Learning rate remains effective
```

Thus, RMSProp can **adapt to changing regions of the loss surface** and is better suited to deep, non-convex optimization.

Advantages

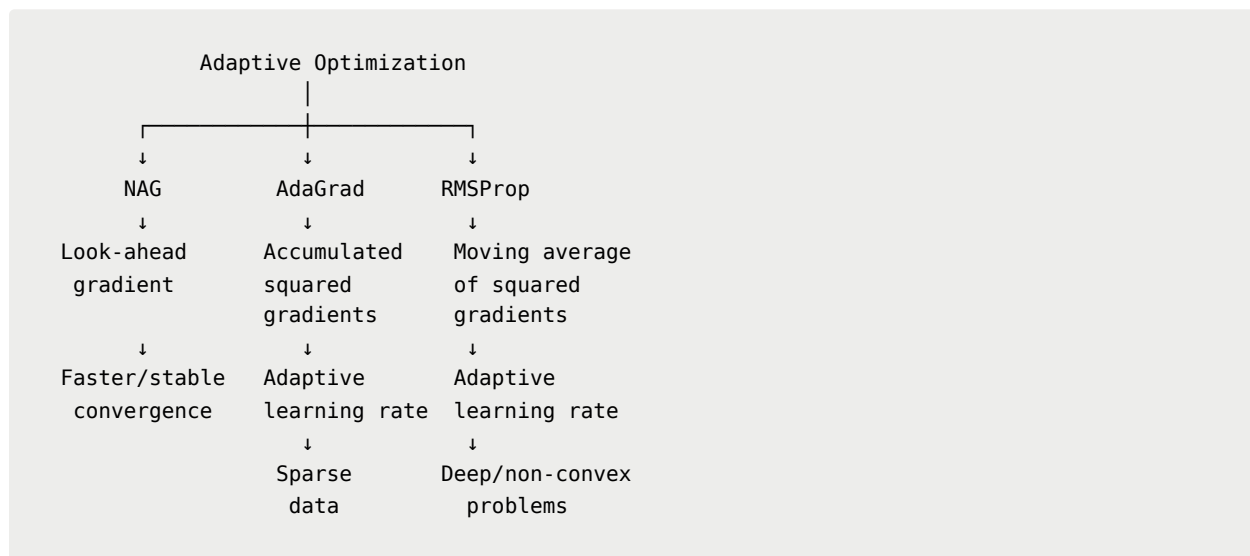
1. Prevents the excessive learning-rate decay of AdaGrad.
2. Provides **adaptive learning rates** for individual parameters.
3. Gives greater importance to **recent gradient information**.
4. Performs well on **deep and non-convex problems**.
5. Helps maintain balanced learning rates throughout training.

5. Comparison

Feature	NAG	AdaGrad	RMSProp
Main idea	Look-ahead momentum	Accumulate squared gradients	Moving average of squared gradients
Uses past information	Momentum/velocity	All past squared gradients	Recent squared gradients
Learning rate	Momentum-based update	Parameter-specific	Parameter-specific
Main benefit	Faster, stable convergence	Good for sparse data	Prevents learning-rate collapse

Feature	NAG	AdaGrad	RMSProp
Major limitation	More computation than simple momentum	Learning rate can become too small	Requires decay hyperparameter
Suitable for	Curved/flat loss surfaces	Sparse features	Deep, non-convex models

6. Overall Working



QUESTION 10:

Explain learning-based approaches to regularization in deep learning. Discuss semi-supervised learning, multi-task learning and sparse representations as regularization techniques.

ANS:

1. Introduction

Learning-based approaches to regularization improve generalization by using additional information, related tasks, or constraints on the learned representation rather than simply restricting the model parameters.

Important learning-based regularization techniques include:

1. Semi-supervised learning

2. Multi-task learning

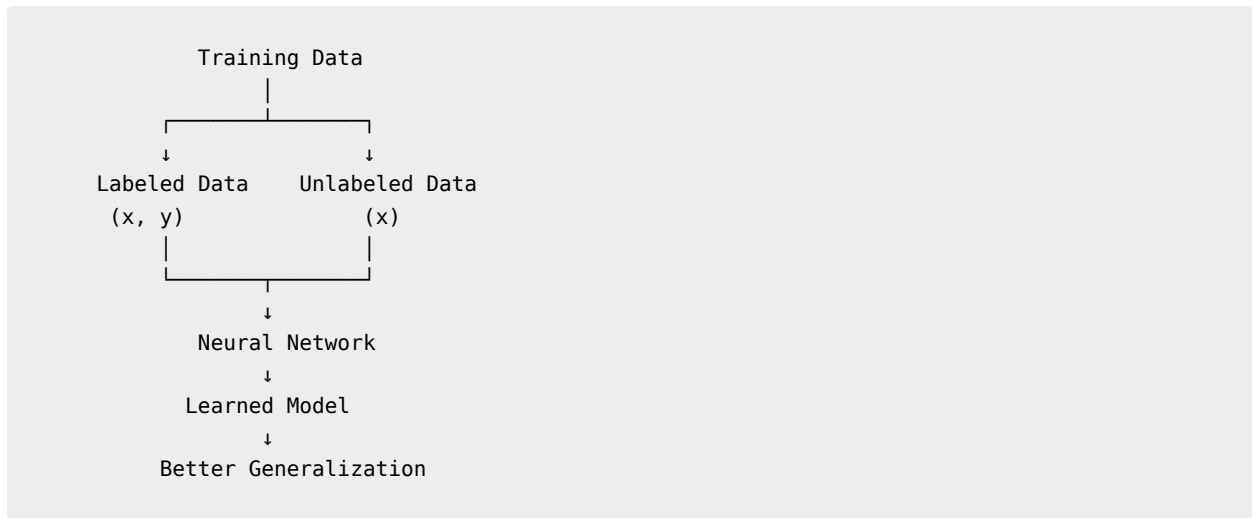
3. Sparse representations

These methods encourage the network to learn **useful and general representations** instead of simply memorizing the training data.

2. Semi-Supervised Learning

Semi-supervised learning uses a combination of **labeled and unlabeled data** for training.

In many practical problems, obtaining labels is expensive, while obtaining raw unlabeled data is relatively easy.



How it acts as regularization

The unlabeled data provides information about the distribution of inputs:

$$P(x)$$

while labeled data provides information about the joint distribution:

$$P(x, y)$$

Using both sources of information places additional constraints on the model and reduces its ability to overfit the limited labeled examples.

Example

Suppose there are:

- **1,000 labeled images**
- **50,000 unlabeled images**

Instead of training only on the 1,000 labeled images, semi-supervised learning can exploit the additional unlabeled images to learn useful structure in the input distribution.

Significance

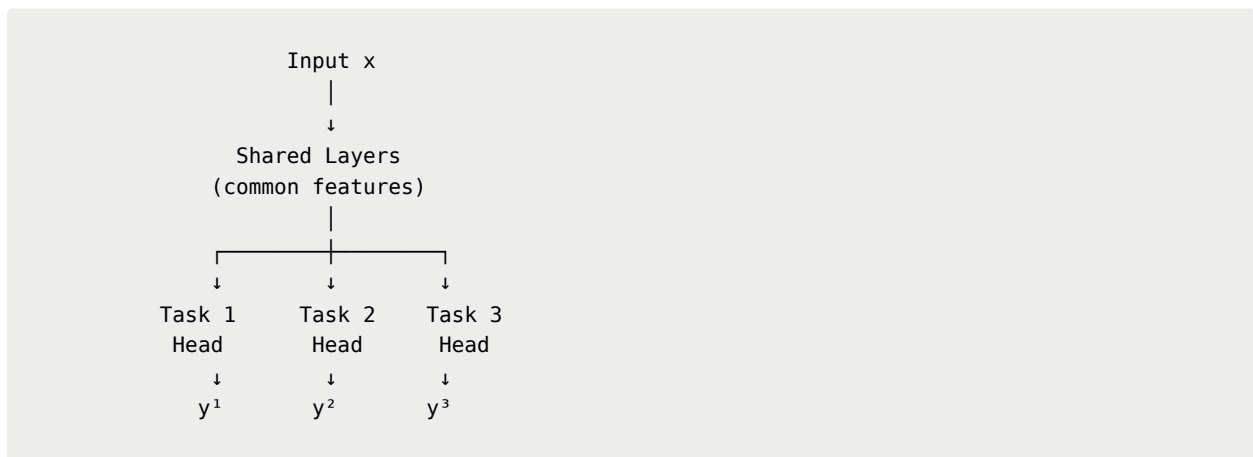
- Reduces dependence on expensive labeled data.
- Makes use of large amounts of **unlabeled data**.
- Helps learn better representations.
- Can improve **generalization** when labeled data is limited.

3. Multi-Task Learning

Multi-task learning (MTL) trains a **single model to perform multiple related tasks simultaneously**.

The important idea is that related tasks **share information**, which acts as a constraint and reduces overfitting to any one task.

Architecture



The **shared layers** learn generic features, while **task-specific layers** produce the outputs for individual tasks.

Example: Face Analysis

For an input face image:

- **Task 1:** Predict age
- **Task 2:** Predict gender
- **Task 3:** Predict emotion

The lower layers can learn common features such as **eyes, nose, shapes and texture**, which are useful for all tasks.

Why it acts as regularization

If the network is trained for only one task, it may learn features specifically suited to the training data of that task.

With multiple related tasks:

Shared Information → **Additional Constraint** → **Less Overfitting** → **Better Generalization**

Thus, the model is encouraged to learn **features that are useful across several tasks** rather than task-specific noise.

4. Sparse Representations

A **sparse representation** expresses a signal, image or dataset using a **small number of non-zero elements** from a larger set of possible features.

A representation is sparse when:

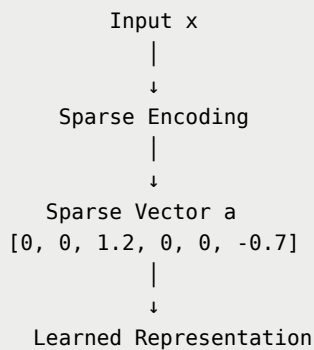
Most coefficients are zero and only a few are non-zero\boxed{\text{Most coefficients are zero and only a few are non-zero}}

The basic idea is:

$$\mathbf{x} \approx \mathbf{D} \mathbf{a}$$

where:

- $\mathbf{x}$ = input signal/data
- $\mathbf{D}$ = **dictionary** containing basis vectors
- $\mathbf{a}$ = sparse coefficient vector.



The dictionary may be **predefined**, such as Fourier or wavelet bases, or **learned from data**, such as in dictionary learning and K-SVD.

5. Sparse Representations in Neural Networks

In neural networks, sparsity can be applied to the **activations** h , rather than only to the model weights θ .

There are two important types:

A. Parameter Sparsity

A penalty is applied directly to model parameters:

Penalty on θ

For example, an **L^1 -norm penalty** can drive many weights to zero.

B. Representational Sparsity

A penalty is applied to the activations:

Penalty on h

This encourages only a small number of neurons to be active for a particular input.

Why it acts as regularization

Sparse representations impose a constraint on what the network can represent. Instead of allowing every feature to participate, the network is encouraged to use only the **most relevant features**.

Therefore:

Sparse Representation → Simpler Features → Less Overfitting → Better Generalization

6. Comparison

Technique	Main Idea	How It Regularizes
Semi-supervised learning	Uses labeled + unlabeled data	Uses additional information from $P(x)P(y)$
Multi-task learning	One model performs multiple related tasks	Shared features constrain each task
Sparse representations	Uses only a few active features	Restricts the complexity of learned representations